Table 3. **Mapping of LLM-based agent capabilities to cyberattack categories.** *Legend:* **High** **Medium** **Low**

Cyberattack Type	Perception	Memory	Reasoning & Planning	Tool Invocation	Multi-agent Collaboration
Reconnaissance and Intelligence					
Threat-Intelligence Gathering	OSINT extraction, IOC mining, KG building	RAG-assisted CVE recall	Threat correlation and prioritisation	SIEM rule generation, API interfacing	Autonomous agent workflow
Exploitation and Payload Delivery					
Penetration Testing	Parsing scan/vuln outputs	Tracking enumeration progress	ReAct planning, attack-graph generation	Automated shell/Nmap/Metaspl calls	Role-decomposed collaboration
Vulnerability Detection	Semantic code/binary parsing	Knowledge-base integration	Cause localisation, patch suggestion	Selective tool orchestration	Cascaded single-agent detector
Malware Generation	Behaviour-to-code conversion	TTP/code pattern memory	Automated payload synthesis	Emit functional malware scripts/code	Autonomous agent swarms
One-/Zero-day Exploitation	Extract CVEs, logs, descriptions	Recall exploit modules/chains	CoT/Reflexive reasoning of paths	Dynamic exploit crafting and parameterisation	Shared roles for recon/exfiltration
Deception and Social Engineering					
Phishing & Social Engineering	Victim profiling from raw text	Contextual memory in dialogue	Psychologically tuned message crafting	Spear-phishing content generation & delivery	Individual agent-driven attack
Honeypot Deployment	Parse attacker input, emulate system response	Track session history, deception context	Adapt interaction strategy based on behaviour	Run realistic shell commands, mimic services	Multi-agent deception or response collaboration
Autonomous Challenge Solving					
Capture-the-Flag Challenges	Problem-statement parsing, flag-pattern recognition	State tracking for multi-step problems	CoT multi-hop reasoning, action planning	Basic decoding/scripting tools	ReAct & Plan single-agent template

analyst effort. Tseng *et al.* [184] push automation into the SOC by chaining GPT-4 tools that extract 2,300 validated IOCs, build relationship graphs, and autogenerate SIEM regexes with 97% accuracy, although post-processing mitigates occasional hallucinations. In the underground-economy domain, Clairoux *et al.* [45] harness GPT-3.5-turbo to summarize 700 cybercrime-forum threads and predict CTI variables with 96.2% overall accuracy, demonstrating LLM versatility with noisy multilingual text.

Benchmarks: Alam *et al.* [14] release CTIBench, a comprehensive APT and malware benchmark. Their evaluation shows GPT-4 leads overall performance while highlighting models' tendency to overestimate threats.

3.1.2 Penetration Testing. In LLM-based penetration-testing agents [104, 136, 171], dynamic reasoning lets agents adapt attack strategies based on discovered vulnerabilities. Initially, LLM-driven penetration testing that still keeps a human "red button" in the loop. Goyal *et al.* [76] first benchmark GPT-3.5-Turbo through GPT-4-Turbo inside pentest workflows, finding that the cheaper model is faster yet loses context in complex attacks. To impose discipline on that

Table 4. The list of LLM-based agent frameworks for cyberattacks. Attack-type abbreviations: CTI = Cyber Threat Intelligence; PT = Penetration Testing; VD = Vulnerability Detection; PSE = Phishing & Social Engineering; MG = Malware Generation; VE = Vulnerability Exploitation; HP = Honeypot Deployment; CTF = Capture the Flag.
Symbols: ✓ = Yes, × = No, △ = Partial, ○ = basic reasoning, ⊙ = advanced, ● = state-of-the-art chain-of-thought.

LLM-based Agents	Attack Type	Params	Context	Open	Multi	Reason	Tool use	Role
MAD-LLM [56]	CTI	varies	8 k	△	×	⊙	AutoGen debate	Purple
LLMCloudHunter [164]	CTI	GPT-4o-V	8 k	×	✓	⊙	Vision & rules	Blue
VulScribeR [49]	CTI	175 B & 7 B	8 k	△	×	○	RAG augmentation	Purple
Crimson [98]	CTI	70 B	16 k	✓	×	●	CVE to ATT&CK	Blue
PentestGPT [52]	PT	backend	16 k	✓	×	⊙	Metasploit CLI	Purple
RapidPen [132]	PT	GPT-4	32 k	×	×	●	RAG executor	Red
Breachseek [19]	PT	GPT-4	128 k	✓	×	⊙	LangGraph planner	Red
Hackphyr [159]	PT	7–13 B	4 k	✓	×	⊙	Internal cmds	Red
AttackLLM [6]	PT	GPT-4	8 k	△	×	⊙	Agent actions	Red
VulnBot [105]	PT	GPT-4o-mini	32 k	✓	×	⊙	Multi-agent	Red
AutoPT [197]	PT	GPT-4	32 k	×	×	⊙	FSM executor	Red
CIPHER [153]	PT	GPT-4	8 k	△	×	○	Function calls	Red
ARACNE [136]	PT	GPT-4	32 k	△	×	⊙	SSH tools	Red
PenHealNet [89]	PT	mixed	8 k	△	×	⊙	Remediation agents	Purple
PenHeal [90]	PT	mixed	8 k	△	×	○	Remediation chain	Purple
LProtector [173]	VD	GPT-4o	128 k	△	✓	●	RAG & CoT	Blue
EvilInstructCoder [87]	VD	7–16 B	4 k	✓	×	○	—	Purple
WitheredLeaf [43]	VD	mixed	8 k	△	×	⊙	Cascade detector	Blue
GRACE [122]	VD	GPT-4	8 k	✓	×	⊙	Graph-aug. prompts	Blue
PDBERT [142]	VD	110 M	512	✓	×	○	—	Blue
PhishAgent [39]	PSE	Otter-MM	4 k	✓	✓	⊙	Vision detector	Blue
ConvoSentinel [8]	PSE	GPT-4	8 k	△	×	⊙	Delegate agents	Blue
SE-OmniGuard [110]	PSE	GPT-4	8 k	△	×	⊙	Persona filter	Blue
WormGPT [70]	PSE	6 B	8 k	△	×	○	—	Red
SEAR [34]	PSE	GPT-4o	128 k	△	✓	⊙	AR interface	Red
AppPoet [218]	MG	GPT-4	8 k	△	×	○	—	Blue
GenTTP [216]	MG	mixed	8 k	✓	×	⊙	Agent parsing	Purple
RedCodeAgent [79]	MG	GPT-4o-mini	32 k	✓	×	⊙	Function calls	Red
SEVENLLM [96]	VE	13 B	8 k	✓	×	⊙	JSON tools	Blue
Net-GPT [151]	VE	hybrid	4 k	✓	×	○	MITM packet gen	Purple
RatGPT [28]	VE	ChatGPT	4 k	△	×	○	Bash shell	Red
AdbGPT [64]	VE	GPT-3.5/4	8 k	✓	×	⊙	ADB automation	Purple
Vul-RAG [57]	VE	GPT-4	32 k	△	×	⊙	RAG	Blue
CVE-LLM [73]	VE	7 B	8 k	✓	×	○	—	Blue
ShelLM [176]	VE	GPT-3.5/4	8 k	✓	×	○	—	Blue
CheatAgent [137]	VE	GPT-3.5/4	8 k	✓	×	⊙	Function calls	Red
ChatIoT [55]	VE	70 B	16 k	✓	×	⊙	RAG	Purple
hackingBuddyGPT [77]	VE	GPT-4	8 k	✓	×	○	Bug-bounty assist	Red
HackerGPT [186]	VE	13 B	4 k	△	×	○	OSINT tools	Red
HoneyLLM [60]	HP	mixed	128 k	×	✓	⊙	Function calls	Blue
LLMPot [187]	HP	4 B/L2/ByT5	8 k	✓	△	⊙	Honeypot sim	Blue
HackSynth [131]	CTF	GPT-4	8 k	△	×	⊙	Plan / summarise	Red
EnIGMA [1]	CTF	GPT-4o	128 k	✓	×	●	GDB / nc tools	Purple

reasoning gap, Wu *et al.* [197] frame each step as a Penetration-Testing State Machine, named AutoPT, which lifted task-completion rates over ReAct [203] while occasionally mis-generating shell commands. In parallel, Pratama *et al.*

[153] fine-tune CIPHER on write-ups for better exploit guidance. Al-Qurishi *et al.* [13] develop PenTest++, an automated framework requiring human oversight.

Happe *et al.* [83] first show GPT-3.5 can guide pentesting when paired with a vulnerable VM, though stability varies. Building on these insights, Deng *et al.* [52] develop PentestGPT, achieving 228.6% better task completion than GPT-3.5. The framework excels at tool usage and output interpretation but struggles with images, strategy selection, and knowledge accuracy. Its three self-interacting modules for penetration testing have gained wide recognition since being open-sourced. Pushing autonomy to enterprise scale, Happe *et al.* [84] fuse hackingBuddyGPT with PentestGPT to compromise an Active Directory lab without operator input, surpassing orchestrators such as MITRE Caldera. Nakatani *et al.* [132] develop RapidPen, a React-driven framework achieving shell access in 200-400s. Huang *et al.* [89] introduce PenHealNet, combining Pentest and Remediation agents to improve upon PentestGPT's capabilities.

Multi-agent penetration testing frameworks coordinate specialized roles for automated security assessments. PenHeal combines testing and remediation to improve coverage by 31% and reduce costs by 46% [90]. Breach-Seek implements a distributed architecture for autonomous scanning [19]. PENTEST-AI integrates MITRE ATT&CK with GPT-4 agents but requires reporting improvements [35]. Furthermore, VulnBot organizes reconnaissance, scanning, and exploitation agents via a penetration-task graph, achieving up to 69% task completion yet still struggles with non-text inputs [105].

Benchmarks: To benchmark the performance of LLM-based agents in automated penetration testing [192], Gioacchini *et al.* [74] introduce AUTOPENBENCH, a 33-task framework spanning access-control, web, network, and cryptography challenges. Complementing this closed-set study, Isozaki *et al.* [94] release an open benchmark driven by PentestGPT and show that LLMs still falter on end-to-end workflows, reinforcing the need for human oversight. Extending the evaluation landscape to CTF environments, Muzsai *et al.* [131] propose HackSynth, whose planner–summarizer architecture solves 41/120 PicoCTF tasks with GPT-4o.

3.1.3 Vulnerability Detection. LLM-based agents can detect vulnerabilities by integrating advanced language perception with structured reasoning and selective tool orchestration, enabling automated, high-fidelity triage across diverse codebases and binary artifacts [173]. Chen *et al.* in [43] propose WitheredLeaf, a cascaded detector that funnels alerts from lightweight language models to GPT-4; across 154 Python and C GitHub projects, it uncovers 123 previously unknown flaws, 45% exploitable, while GPT-4's 60% success on synthetic EIBs is bolstered by CodeBERT and Code Llama to sharpen recall and trim false positives. Building on this, Hossen *et al.* [87] present EvilInstructCoder, revealing that poisoning just 0.5% of instruction-tuning data for code LLMs yields 76–86% attack success, spotlighting urgent defence gaps as code LLMs permeate development pipelines. Complementing these insights, Akuthota *et al.* [10] report a 77% accuracy from GPT-3.5-Turbo on 2,740 snippets spanning SQLi, XSS, and command-injection.

Recent advancements in RAG and structure-aware LLM-based agents have demonstrated significant improvements in C/C++ and binary code vulnerability detection through enhanced accuracy, expanded training datasets, and optimized resource utilization. For instance, LProtector [173] demonstrates the effectiveness of integrating GPT-4o with RAG and CoT reasoning, achieving 89.68% accuracy and 33.49% F1 scores on 5,000 balanced Big-Vul samples, outperforming established tools while identifying limitations in plain text code processing. VulScribeR [49] presents an innovative approach to dataset enhancement, leveraging ChatGPT-3.5 and CodeQwen-1.5 with specialized prompting techniques to generate 1,000 vulnerability examples cost-effectively at US $1.88, resulting in F1 score improvements of up to 30.80% across multiple datasets, though effectiveness depends on prompt optimization. Additionally, GRACE [122] incorporates graph-based contextual demonstrations, demonstrating superior performance with a 28.65% F1 improvement across

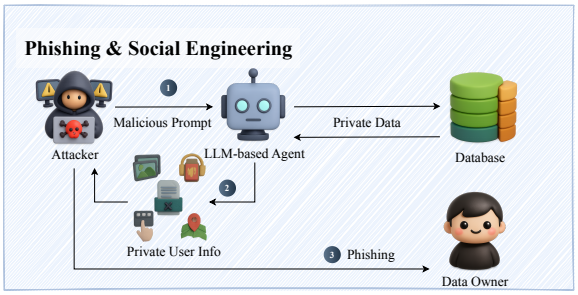

Fig. 4. LLM-based agents' cyberattack capabilities of phishing and social engineering.

comparable datasets, despite current C/C++ limitations. Furthermore, PDBERT by Panebianco *et al.* [142] reveals critical insights regarding model limitations.

3.1.4 Phishing and Social Engineering. As shown in Fig. 4, LLMs can craft convincing phishing emails, chats, and voice scripts, making social engineering harder to detect. Using victim-specific language and automated workflows, these agents transform manual campaigns into instant, personalized attacks at scale [71]. For instance, Alotaibi *et al.* [18] demonstrate that prompt-engineering can bypass safeguards to mass-produce phishing content cheaper than humans, while surveying countermeasures and deepfake risks. Furthermore, Begou *et al.* [29] show ChatGPT can deploy complete phishing kits in 10 minutes, noting token limits and withholding specifics. Roy *et al.* [163] analyze how attackers bypass ChatGPT, Claude, and Bard safeguards, proposing prompt-level detection. Finally, Chen *et al.* [42] introduce PEN using LLMs to synthesize novel phishing samples.

LLMs can automate and scale phishing creation, subsequent work pivots to testing the resilience of current defenses, probing new AI-mediated attack channels, and proposing multimodal countermeasures [4]. Figueiredo *et al.* [69] propose ViKing system that uses GPT and voice modules to persuade 52% of participants to divulge sensitive data, with 71.25% rating its replies effective. Cao *et al.* [39] design PhishAgent that achieves 94% detection accuracy while resisting brand-obfuscation attacks. Finally, Yang *et al.* [201] uncover fox8, a network of 1,140 ChatGPT-assisted Twitter bots whose interaction patterns defeat standard detectors, highlighting the need for models trained on real adversarial data.

In addition to phishing, LLM-enabled social engineering agents now focus on intent-driven conversational attacks based on psychological analysis and evaluation. Yu *et al.* [207] classify AI-driven social engineering through their taxonomy, reviewing 117 studies and developing a Markov process to measure penetration efficiency and costs.

Benchmarks: Evaluating these threats, Ai *et al.* [8] create SEConvo with 5,300 dialogues and ConvoSentinel pipeline, which increases F1 scores by 12% against LLM-generated attacks. Kumarage *et al.* [110] develop SE-VSim to simulate 1,350 persona-based conversations and SE-OmniGuard to improve detection by 8-15%.

3.2 Automated Weaponization

3.2.1 Malware Generation. As shown in Fig. 5, LLM-based agents in cybersecurity enable automated malware generation through code generation [86, 97]. Using natural language programming and script generation, agents convert behavioral descriptions to attack code, evade detection, and generate variant malware. This language-code fusion accelerates malware development while expanding attack potential with minimal human input. The authors in [70]

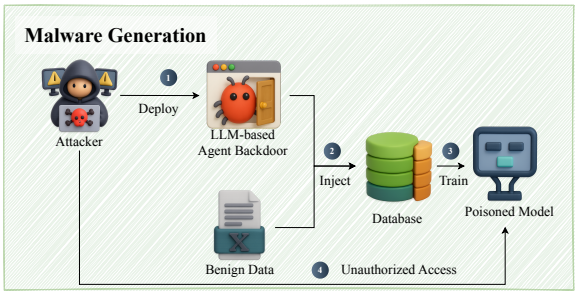

Fig. 5. LLM-based agents' cyberattack capabilities of malware generation.

present what they describe as the first peer-reviewed study of WormGPT, a black-hat LLM built on EleutherAI's GPT-J and trained on malware-related data, after an exhaustive literature search revealed no prior academic work on the subject. Charan *et al.* in [40] examine the malicious use of LLMs for generating cyberattack payloads, analyzing over 500,000 real-world malware samples from 2022 and systematically producing executable code for the top 10 MITRE Techniques. Their findings show that ChatGPT outperforms Bard in producing coherent, functional code and handling error resolution, thereby enabling the development of more sophisticated attack vectors. Transitioning from payload synthesis to behavioral analysis, Zhang *et al.* [216] introduce GENTTP for extracting TTPs from malware. The model was tested on labeled and real-world datasets. GPT-4 achieved 0.90 coverage and 0.99 sequence accuracy. GENTTP surpassed other LLMs in detecting behavioral patterns. Building on this foundation, Patsakis *et al.* [148] and Lin *et al.* [118] explore LLM performance in script-level deobfuscation, focusing on PowerShell samples from the Emotet malware campaign. Extending beyond individual samples.

Building upon prior concerns surrounding LLM-enabled cyber threats, Beckerich *et al.* in [28] examine LLMs as malware proxies. Their POC shows ChatGPT enabling covert command and control (C2) communication through plugin exploitation. Extending the discussion from offensive misuse to defensive innovation, Zhao *et al.* in [218] introduce AppPoet, a multi-view LLM-based Android malware detection system that leverages GPT-4 for generation and text-embedding-ada-002 for embedding tasks. AppPoet integrates static feature extraction with human-readable behavioral analysis and utilizes a DNN classifier to combine multi-view data. When tested on a dataset of 11,189 benign apps from AndroZoo and 12,128 malware samples verified via VirusTotal, the system achieved a detection accuracy of 97.15% and an F1 score of 97.21%.

Benchmarks: Transitioning to safe development environments, Guo *et al.* [78] created RedCode to test code agent safety. They evaluated multiple agents across thousands of tests in sandboxed environments. Their findings show GPT-4 produces more harmful code despite safeguards. Building on this, Guo *et al.* [79] developed RedCodeAgent, achieving 72.47% attack success rate, which highlights the need for better automated safety testing.

3.2.2 Vulnerability Exploitation: One-Day and Zero-Day Attacks. LLM-based agents with code analysis and reasoning abilities enable autonomous systems to detect and exploit software vulnerabilities dynamically. Through semantic analysis, exploit chain construction, and automated tool integration, these agents transform manual exploitation into rapid, adaptive workflows. Studies show their effectiveness across cloud, web, and mobile environments, demonstrating their ability to expand attack coverage with reduced expertise requirements. As shown in Fig. 6, Patil *et al.* [147] inaugurate this discourse on the defensive front by showing that LLM-powered anomaly detectors improve zero-day spotting in